Requirement :- MATLAB Software and Code Composer Studio

Theory :- An FFT computes the DFT and produces exactly the same result as evaluating the DFT definition directly; the most important difference is that an FFT is much faster. (In the presence of round-off error, many FFT algorithms are also much more accurate than evaluating the DFT definition directly, as discussed below.)

Let $x_0, \ldots, x_{N-1}$ be complex numbers. The DFT is defined by the formula

$$X_k = \sum_{n=0}^{N-1} x_n \exp^{-i2\pi kn/N} \qquad k = 0, 1 \cdots, N-1$$

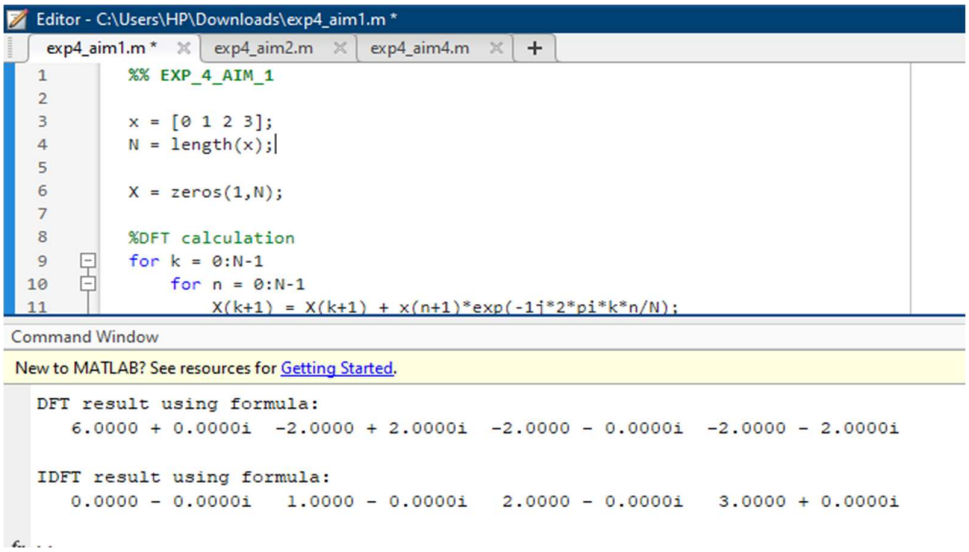
Evaluating this definition directly requires $O(N^2)$ operations: there are N outputs X[k], and each output requires a sum of N terms. An FFT is any method to compute the same results in O(N log N) operations.

Aim 1: Find the DFT and IDFT of the given sequence using definitions. Say, x[n] = {0, 1, 2, 3}

Code :-

```
%% EXP_4_AIM_1

x = [0 1 2 3];
N = length(x);

X = zeros(1,N);

%DFT calculation
for k = 0:N-1
    for n = 0:N-1
        X(k+1) = X(k+1) + x(n+1)*exp(-1j*2*pi*k*n/N);
    end
end

%IDFT calculation
x_new = zeros(1,N);

for n = 0:N-1
    for k = 0:N-1
        x_new(n+1) = x_new(n+1) + X(k+1)*exp(1j*2*pi*k*n/N);
    end
end

disp('DFT result using formula:')
disp(X)
disp('IDFT result using formula:')
disp((1/4)*x_new)
```

Result :-



Conclusion :-

This experiment demonstrates the manual implementation of the Discrete Fourier Transform (DFT) and its corresponding Inverse Discrete Fourier Transform (IDFT) for a finite discrete-time sequence. The DFT was computed using nested loops based on its fundamental definition, converting the given time-domain signal into its frequency-domain representation. Subsequently, the IDFT was implemented to reconstruct the original signal from its frequency components. After applying the required normalization factor, the reconstructed output matches the original input sequence, confirming the correctness of the implementation. The results verify that DFT and IDFT form an exact transform pair and that the original signal can be perfectly recovered from its frequency representation, with only negligible numerical variations due to computational precision.

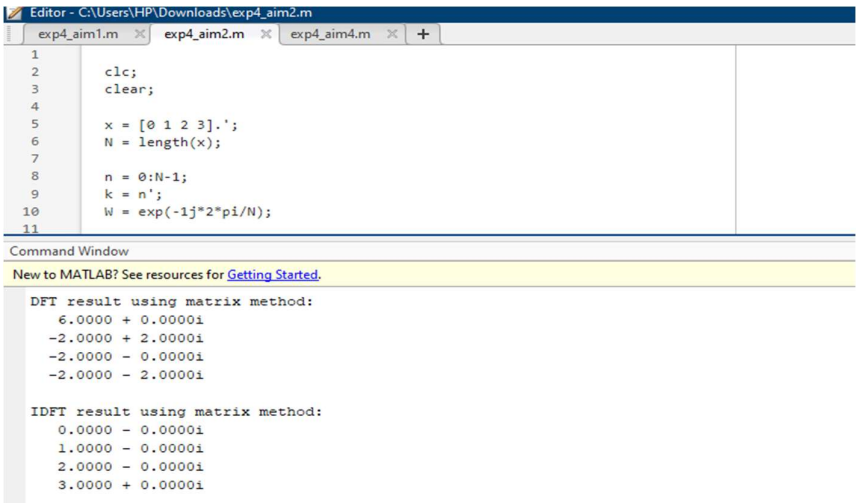
Aim 2: Repeat Aim 1 using compact matrix representation.

Code :-

```
clc;
clear;

x =[0 1 2 3];
N = length(x);

n = 0:N-1;
k = n';
W = exp(-1j*2*pi/N);

F = W.^(k*n);

X = F * x;
disp('DFT result using matrix method:')
disp(X)
```

```
F_inv = (1/N) * conj(F);

x_reconstructed = F_inv * X;

disp('IDFT result using matrix method:')
disp(x_reconstructed)
```

Result :-



Conclusion :-

The Discrete Fourier Transform (DFT) and Inverse Discrete Fourier Transform (IDFT) of the sequence (x = [1, 2, 3, 4]) are computed using a compact matrix representation approach. In this method, a DFT matrix is constructed using complex exponential basis functions derived from the twiddle factor (e^{-j2piN}), where (N) is the length of the sequence. The DFT is obtained by performing matrix multiplication between the DFT matrix and the input column vector. The IDFT is computed by multiplying the transformed sequence with the scaled conjugate of the DFT matrix, ensuring accurate reconstruction of the original signal. This matrix-based implementation provides a systematic and mathematically structured way to compute the DFT and IDFT, highlighting the linear transformation nature of the Fourier transform in discrete-time signal processing.

Aim 3: Consider Aim 1 and Aim 2 and write "C" program to verify using Code Composer Studio (CCS-3.1) IDE and the DSP TMS 320C6713 DSK Kit.

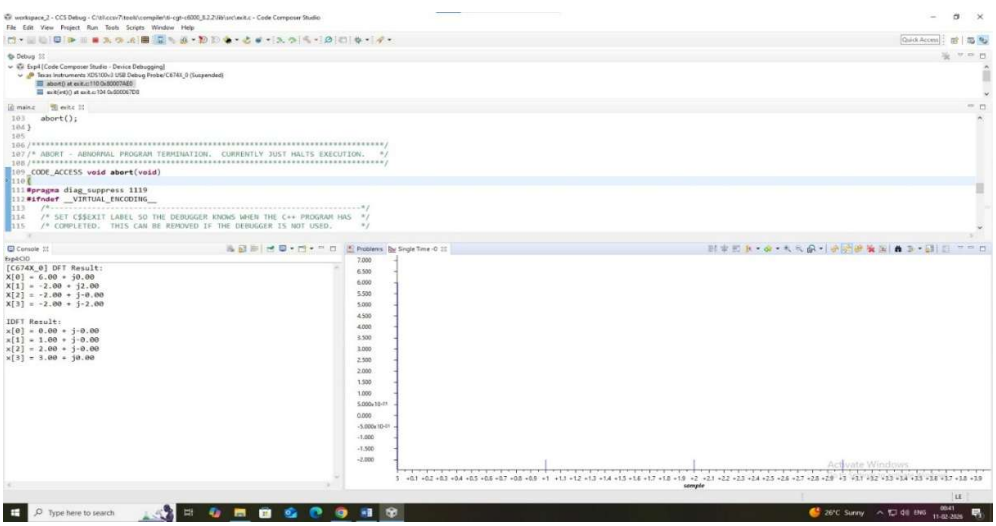
Code :-

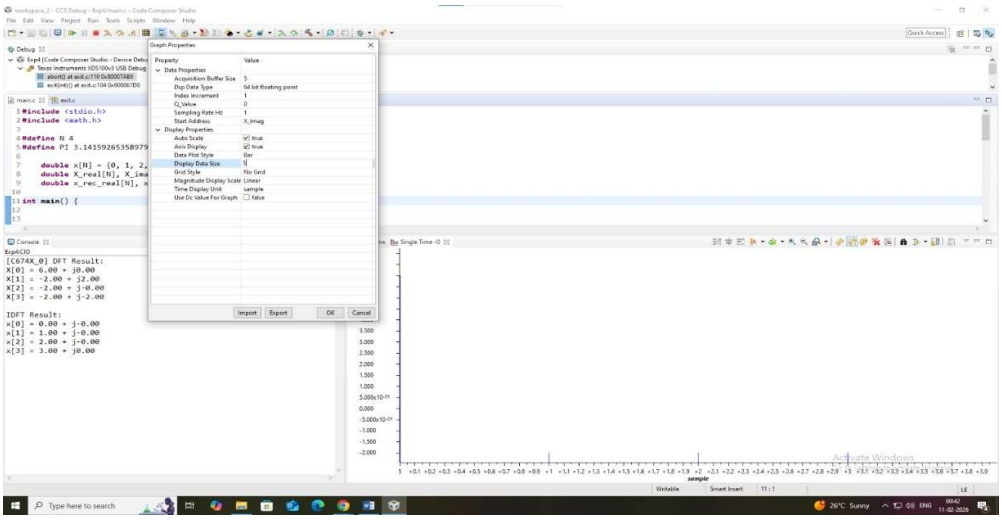
```
#include <stdio.h>

#include <math.h>
```

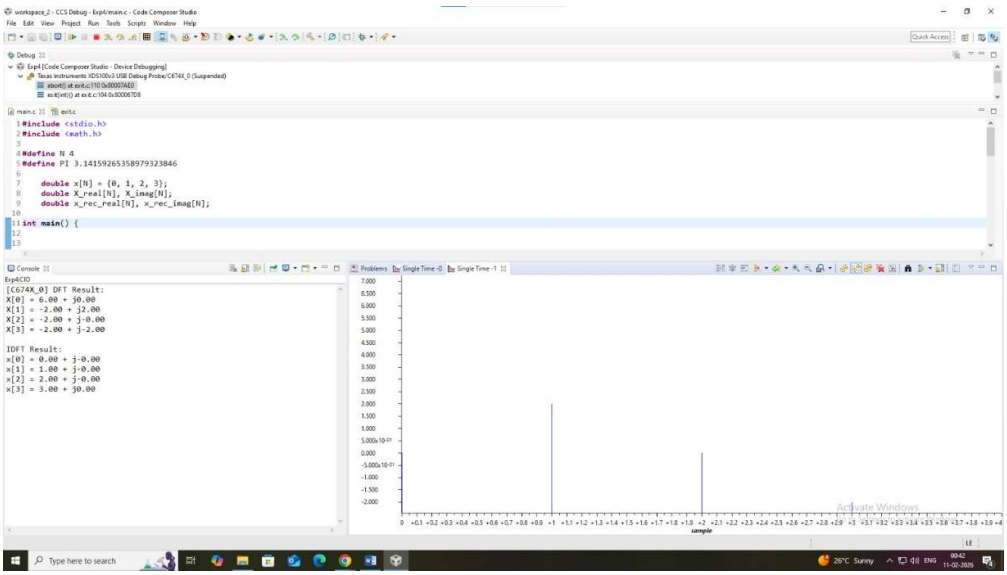
```
#define N 4
#define PI 3.14159265358979323846

int main()
{
   int n, k;

   double x_real[N] = {0,1,2,3};                    // Input sequence
   double x_imag[N] = {0, 0, 0, 0};

   double X_real[N] = {0}, X_imag[N] = {0};                    // DFT result
   double xr_real[N] = {0}, xr_imag[N] = {0};                    // IDFT result

   double angle, c, s;

   /* DFT computation */
   for(k = 0; k < N; k++)
   {
      for(n = 0; n < N; n++)
      {
         angle = -2 * PI * k * n / N;
         c = cos(angle);
         s = sin(angle);
         X_real[k] += x_real[n]*c - x_imag[n]*s;
         X_imag[k] += x_real[n]*s + x_imag[n]*c;
      }
   }

   /* IDFT computation */
   for(n = 0; n < N; n++)
   {
      for(k = 0; k < N; k++)
```

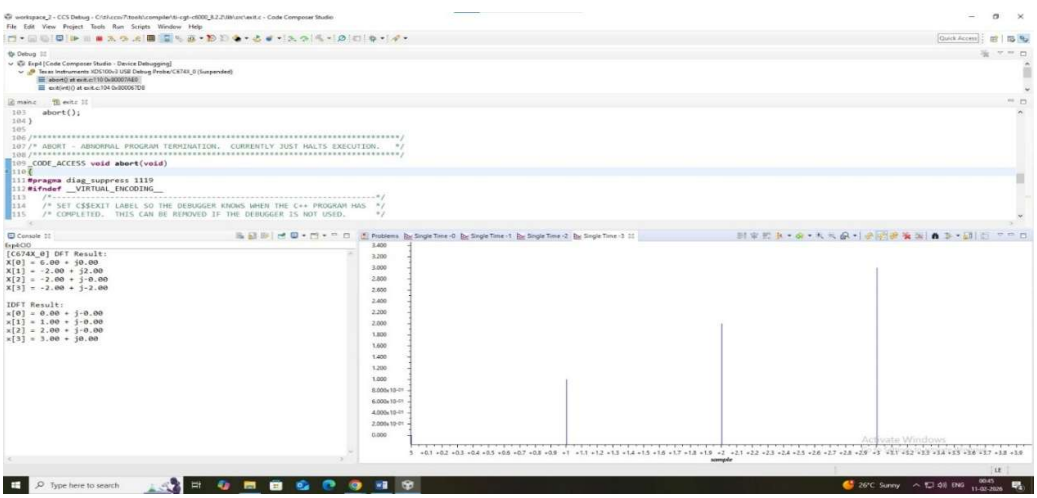
```
        {
            angle = 2 * PI * k * n / N;
            c = cos(angle);
            s = sin(angle);
            xr_real[n] += X_real[k]*c - X_imag[k]*s;
            xr_imag[n] += X_real[k]*s + X_imag[k]*c;
        }
        xr_real[n] /= N;
        xr_imag[n] /= N;
    }
    while(1);
    return 0;
}
```

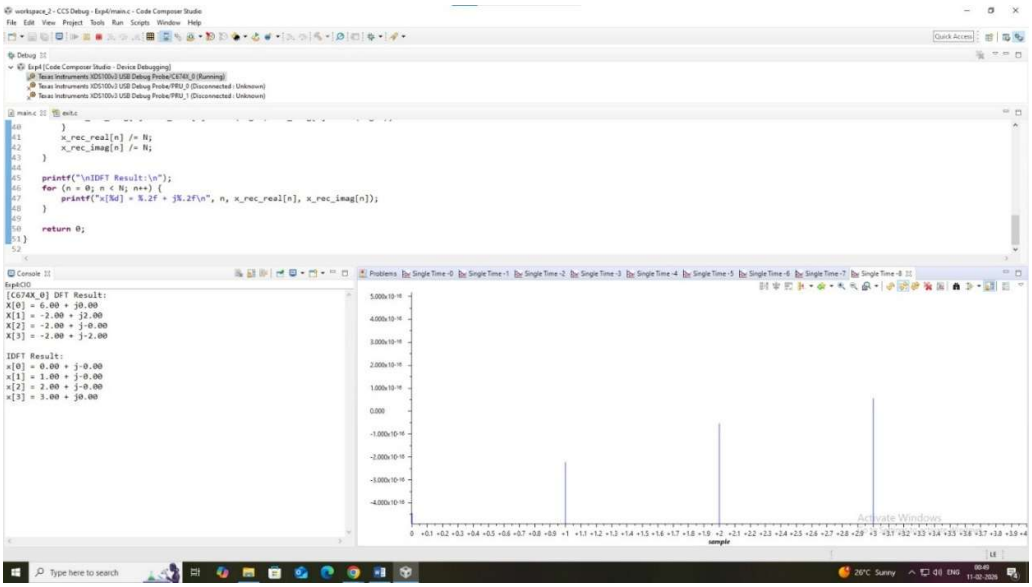
Result :-




workspace_2 - CCS Debug - Exp4/main.c - Code Composer Studio
File Edit View Project Run Tools Scripts Window Help
Graph Properties
Property
Value
Data Properties
Acquisition Buffer Size
Dsp Data Type
Index Increment
Q Value
Sampling Rate Hz
Start Address
Display Properties
Auto Scale
Axis Display
Data Plot Style
Display Data Size
Grid Style
Magnitude Display Scale
Time Display Unit
Use Dc Value For Graph
Import
Export
OK
Cancel
#include <stdio.h>
#include <math.h>
#define N 4
int main() {
[C674X_0] DFT Result:
X[0] = 6.00 + j0.00
X[1] = -2.00 + j2.00
X[2] = -2.00 + j-0.00
X[3] = -2.00 + j-2.00
IDFT Result:
x[0] = 0.00 + j-0.00
x[1] = 1.00 + j-0.00
x[2] = 2.00 + j-0.00
x[3] = 3.00 + j0.00
sample


workspace_2 - CCS Debug - Exp4/main.c - Code Composer Studio
#define N 4
#define PI 3.14159265358979323846
double x[N] = {0, 1, 2, 3};
double X_real[N], X_imag[N];
double x_rec_real[N], x_rec_imag[N];
int main() {
[C674X_0] DFT Result:
X[0] = 6.00 + j0.00
X[1] = -2.00 + j2.00
X[2] = -2.00 + j-0.00
X[3] = -2.00 + j-2.00
IDFT Result:
x[0] = 0.00 + j-0.00
x[1] = 1.00 + j-0.00
x[2] = 2.00 + j-0.00
x[3] = 3.00 + j0.00
sample


abort();
/* ABORT - ABNORMAL PROGRAM TERMINATION. CURRENTLY JUST HALTS EXECUTION. */
_CODE_ACCESS void abort(void)
#pragma diag_suppress 1119
#ifndef __VIRTUAL_ENCODING__
/* SET C$$EXIT LABEL SO THE DEBUGGER KNOWS WHEN THE C++ PROGRAM HAS */
/* COMPLETED. THIS CAN BE REMOVED IF THE DEBUGGER IS NOT USED. */
[C674X_0] DFT Result:
X[0] = 6.00 + j0.00
X[1] = -2.00 + j2.00
X[2] = -2.00 + j-0.00
X[3] = -2.00 + j-2.00
IDFT Result:
x[0] = 0.00 + j-0.00
x[1] = 1.00 + j-0.00
x[2] = 2.00 + j-0.00
x[3] = 3.00 + j0.00
sample



Conclusion :-

In this experiment, the Discrete Fourier Transform (DFT) and Inverse Discrete Fourier Transform (IDFT) were successfully implemented in C and verified using **Code Composer Studio** on the **TMS320C6713 DSK**. The program computed the frequency-domain representation of the input sequence and then reconstructed the original time-domain signal using the IDFT algorithm.The computed DFT results were stored in separate real and imaginary arrays, and the IDFT results correctly reproduced the original input sequence after normalization by (1/N). The successful reconstruction confirms the mathematical correctness of the implementation and validates the theoretical relationship between DFT and IDFT.Thus, the experiment demonstrates practical realization of Fourier transform algorithms on a DSP processor and verifies their correctness using embedded implementation tools.

Aim 4: To find N point FFT and IFFT of the given sequence (Decimation in time and decimation in frequency algorithms) without using the MATLAB inbuilt functions. Plot and check your results with the inbuilt functions.

Code :-

```
x=[0 1 2 3];
N=length(x);

x=bitrevorder(x);

%DIT FFT
for s=1:log2(N)
    m=2^s;
```

```
    wm=exp(-1j*2*pi/m);
    for k=1:m:N
        w=1;
        for j=0:m/2-1
            t=w*x(k+j+m/2);
            u=x(k+j);
            x(k+j)=u+t;
            x(k+j+m/2)=u-t;
            w=w*wm;
        end
    end
end

disp('DIT-FFT result:')
X_fft=x

disp('DIT-FFT using inbuit method result:')
X_builtin=fft([0 1 2 3])


%% EXP_4_AIM_4_PART_2

x=[6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 - 0.0000i  -2.0000 - 2.0000i];
N=length(x);

%DIT IFFT
for s=log2(N):-1:1
    m=2^s;
    wm=exp(1j*2*pi/m);

    for k=1:m:N
        w=1;
        for j=0:m/2-1
            u=x(k+j);
            v=x(k+j+m/2);
            x(k+j)=u+v;
            x(k+j+m/2)=(u-v)*w;
            w=w*wm;
        end
    end
end

x_dit=bitrevorder(x);

disp('The DIT-IFFT result is ')
x_dit = x_dit/4

disp('DIT-IFFT using inbuit method result:')
X_builtin = ifft([6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 - 0.0000i  -2.0000
- 2.0000i])



%% EXP_4_AIM_4_PART_3

x=[0 1 2 3];
N=length(x);

%DIF FFT
```

```
for s=log2(N):-1:1
    m=2^s;
    wm=exp(-1j*2*pi/m);

    for k=1:m:N
        w=1;
        for j=0:m/2-1
            u=x(k+j);
            v=x(k+j+m/2);
            x(k+j)=u+v;
            x(k+j+m/2)=(u-v)*w;
            w=w*wm;
        end
    end
end

x_dit=bitrevorder(x);

disp('The DIF-FFT result is ')
x_dit

disp('The DIF-FFT using inbuilt method result is ')
X_builtin = fft([0 1 2 3])


%% EXP_4_AIM_4_PART_4

x=[6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 - 0.0000i  -2.0000 - 2.0000i];
N=length(x);

x=bitrevorder(x);

%DIT FFT
for s=1:log2(N)
    m=2^s;
    wm=exp(1j*2*pi/m);
    for k=1:m:N
        w=1;
        for j=0:m/2-1
            t=w*x(k+j+m/2);
            u=x(k+j);
            x(k+j)=u+t;
            x(k+j+m/2)=u-t;
            w=w*wm;
        end
    end
end

disp('The DIF-IFFT result is ')
X_fft=x/4


disp('The result using DIF-IFFT inbuit method is ')
X_builtin = ifft([6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 - 0.0000i  -2.0000
- 2.0000i])
```

Result :-

```
DIF-FFT result:

x_dif =

   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 + 0.0000i  -2.0000 - 2.0000i

DIF-FFT using inbuilt method result:

X_builtin =

   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 + 0.0000i  -2.0000 - 2.0000i

DIF-IFFT result:

X_fft =

   0.0000 + 0.0000i   1.0000 + 0.0000i   2.0000 + 0.0000i   3.0000 - 0.0000i

DIF-IFFT using inbuilt method:

X_builtin =

     0     1     2     3
```

```
DIT-IFFT result:

x_dit =

   0.0000 + 0.0000i   1.0000 + 0.0000i   2.0000 + 0.0000i   3.0000 - 0.0000i

DIT-IFFT using inbuilt method result:

X_builtin =

     0     1     2     3

DIF-FFT result:

x_dif =

   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 + 0.0000i  -2.0000 - 2.0000i

DIF-FFT using inbuilt method result:

X_builtin =

   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 + 0.0000i  -2.0000 - 2.0000i

DIF-IFFT result:

X_fft =

   0.0000 + 0.0000i   1.0000 + 0.0000i   2.0000 + 0.0000i   3.0000 - 0.0000i
```

```
DFT result using formula:
   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 - 0.0000i  -2.0000 - 2.0000i

IDFT result using formula:
   0.0000 - 0.0000i   1.0000 - 0.0000i   2.0000 - 0.0000i   3.0000 + 0.0000i

DFT result using matrix method:
   6.0000 + 0.0000i
  -2.0000 + 2.0000i
  -2.0000 - 0.0000i
  -2.0000 - 2.0000i

IDFT result using matrix method:
   0.0000 - 0.0000i
   1.0000 - 0.0000i
   2.0000 - 0.0000i
   3.0000 + 0.0000i

DIT-FFT result:

X_fft =

   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 + 0.0000i  -2.0000 - 2.0000i

DIT-FFT using inbuilt method result:

X_builtin =

   6.0000 + 0.0000i  -2.0000 + 2.0000i  -2.0000 + 0.0000i  -2.0000 - 2.0000i
```

Conclusion :-

In this experiment, the Fast Fourier Transform (FFT) and Inverse Fast Fourier Transform (IFFT) were implemented using both Decimation-In-Time (DIT) and Decimation-In-Frequency (DIF) algorithms for a 4-point sequence. The butterfly computations were carried out stage by stage using twiddle factors, and bit-reversal ordering was applied wherever required to obtain results in proper sequence.The manually implemented DIT-FFT and DIF-FFT algorithms produced frequency-domain results identical to MATLAB's inbuilt fft() function, verifying the correctness of the butterfly structure and twiddle factor computations. Similarly, the DIT-IFFT and DIF-IFFT implementations successfully reconstructed the original time-domain sequence after proper scaling by (1/N), matching the output of MATLAB's ifft() function.Thus, the experiment confirms that both DIT and DIF approaches are computationally efficient methods for evaluating the DFT and IDFT, significantly reducing computational complexity from (O(N^2)) to (O(N log N)). The results validate the theoretical principles of FFT algorithms and demonstrate their accuracy and practical implementation in MATLAB.

Aim 5: Consider Aim 4 and write "C" program to verify using Code Composer Studio (CCS-3.1) IDE and the DSP TMS 320C6713 DSK Kit.

Code :-

```
#include <stdio.h>

#include <math.h>


#define N 4

#define PI 3.14159265358979323846
```

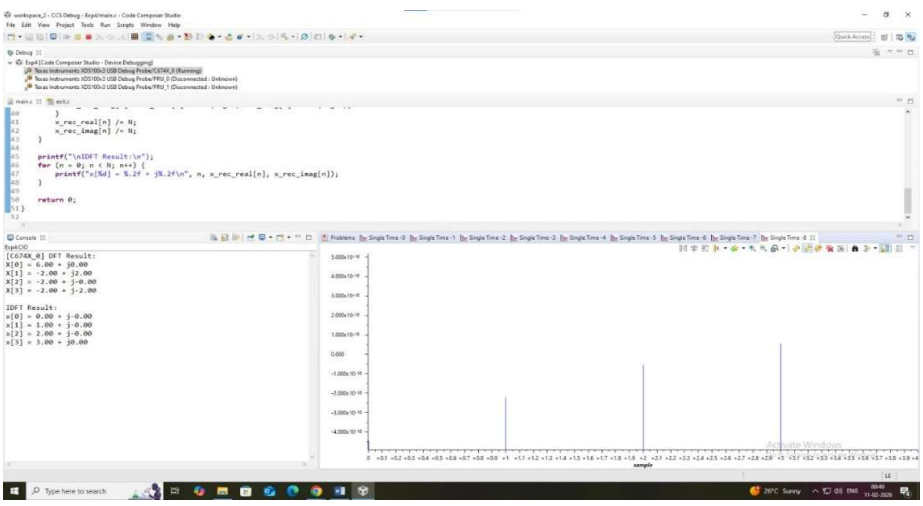
```
void bit_reverse(double xr[], double xi[])
{
    int i, j = 0, k;
    for(i = 0; i < N; i++)
    {
        if(i < j)
        {
            double tr = xr[i];
            double ti = xi[i];
            xr[i] = xr[j];
            xi[i] = xi[j];
            xr[j] = tr;
            xi[j] = ti;
        }
        k = N >> 1;
        while(k <= j)
        {
            j -= k;
            k >>= 1;
        }
        j += k;
    }
}

int main()
{
    int i, j, k, stage, m;
    double wr, wi, tr, ti, ur, ui;
    double xr[N] = {0,1,2,3};
    double xi[N] = {0,0,0,0};
```

```
bit_reverse(xr, xi);

for(stage = 1; stage <= 2; stage++)
{
   m = 1 << stage;
   double wm_r = cos(-2*PI/m);
   double wm_i = sin(-2*PI/m);

   for(k = 0; k < N; k += m)
   {
      wr = 1.0;
      wi = 0.0;

      for(j = 0; j < m/2; j++)
      {
         tr = wr * xr[k+j+m/2] - wi * xi[k+j+m/2];
         ti = wr * xi[k+j+m/2] + wi * xr[k+j+m/2];

         ur = xr[k+j];
         ui = xi[k+j];
         xr[k+j] = ur + tr;
         xi[k+j] = ui + ti;
         xr[k+j+m/2] = ur - tr;
         xi[k+j+m/2] = ui - ti;
        double temp = wr;
         wr = temp*wm_r - wi*wm_i;
         wi = temp*wm_i + wi*wm_r;
      }
   }
}

double Xr[N], Xi[N];
```

```
 for(i=0;i<N;i++)
 {
    Xr[i]=xr[i];
    Xi[i]=xi[i];
 }
bit_reverse(Xr, Xi);
for(stage = 1; stage <= 2; stage++)
 {
    m = 1 << stage;
    double wm_r = cos(2*PI/m);
    double wm_i = sin(2*PI/m);
    for(k = 0; k < N; k += m)
    {
       wr = 1.0;
       wi = 0.0;

       for(j = 0; j < m/2; j++)
       {
          tr = wr * Xr[k+j+m/2] - wi * Xi[k+j+m/2];
          ti = wr * Xi[k+j+m/2] + wi * Xr[k+j+m/2];
         ur = Xr[k+j];
          ui = Xi[k+j];
           Xr[k+j] = ur + tr;
          Xi[k+j] = ui
          Xr[k+j+m/2] = ur - tr;
          Xi[k+j+m/2] = ui - ti;
          double temp = wr;
          wr = temp*wm_r - wi*wm_i;
          wi = temp*wm_i + wi*wm_r;
       }
    }
 }
```

```
for(i=0;i<N;i++)
{
    Xr[i] = Xr[i]/N;
    Xi[i] = Xi[i]/N;
}
 while(1);
return 0; }
```

Result :-



Conclusion :-

In this experiment, the Fast Fourier Transform (FFT) and Inverse Fast Fourier Transform (IFFT) were implemented using the Decimation-In-Time (DIT) algorithm in C and verified using **Code Composer Studio** on the **TMS320C6713 DSK**. The program performed butterfly computations stage by stage along with bit-reversal reordering to obtain the correct frequency-domain output.

The FFT computation successfully transformed the time-domain input sequence into its frequency-domain representation, and the IFFT reconstructed the original signal after proper normalization by (1/N). The reconstructed output matched the original input sequence, confirming the correctness of the implementation.Thus, the experiment validates the theoretical principles of the FFT algorithm and demonstrates its practical realization on a DSP processor. It also highlights the computational efficiency of FFT, reducing complexity from (O(N^2)) to (O(N log N)), making it highly suitable for real-time signal processing applications.